JavaScript Array Traversal using `for...of`

The `for...of` **loop** is a modern JavaScript loop introduced in **ES6**. It is used to iterate directly over the values of an array.

Unlike the `for` loop, you do not need to use an index.

Table of Contents

What is `for...of` ?

Definition

The `for...of` loop iterates over the **values** of an iterable object such as an array.

Why Use `for...of` ?

We use `for...of` to:

- Traverse arrays easily
- Avoid using indexes
- Write cleaner code
- Improve readability

Syntax

```
for(const element of array){

    // code

}
```

Example 1 (Print Array)

```
const fruits = [

    "Apple",

    "Mango",

    "Orange"

];

for(const fruit of fruits){

    console.log(fruit);

}
```

Output

```
Apple
Mango
Orange
```

Example 2 (Numbers)

```
const numbers = [
    10,
    20,
    30
];
for(const number of numbers){
    console.log(number);
}
```

Output

```
10
20
30
```

Example 3 (Sum of Numbers)

```
const numbers = [
    10,
    20,
    30
];
let sum = 0;
for(const number of numbers){
    sum += number;
```

```
}

console.log(sum);
```

Output

```
60
```

Example 4 (Objects)

```
const users = [
    {
        name: "John"
    },
    {
        name: "Sara"
    }
];
for(const user of users){
    console.log(user.name);
}
```

Output

```
John
```

```
Sara
```

Real Project Example

Shopping Cart

```
const cart = [
    "Laptop",
    "Mouse",
    "Keyboard"
];
for(const item of cart){
    console.log(item);
}
```

Best Practices

- Use `for...of` when you only need array values.
- Use meaningful variable names.
- Prefer `const` if the value will not change.

Common Mistakes

Mistake 1

Trying to get the index.

Wrong

```
for(const item of array){
    console.log(item.index);
}
```

`for...of` gives values, **not indexes**.

Mistake 2

Using `for...in` instead of `for...of` .

```
for...in
```

returns indexes.

```
for...of
```

returns values.

Interview Questions

What does `for...of` iterate over?

Array values.

Does `for...of` provide indexes?

No.

Which version of JavaScript introduced `for...of` ?

ES6.

Summary Table

Feature	Description
Loop	`for...of`
Iterates Over	Values
Index Available	No
Introduced In	ES6
Best Use	Simple array traversal

Final Notes

The `for...of` loop is cleaner and easier to read than the traditional `for` loop when you only need the array values. It is widely used in modern JavaScript applications.

JavaScript Array Traversal using `for...of`

The `for...of` **loop** is a modern JavaScript loop introduced in **ES6**. It is used to iterate directly over the values of an array.

Unlike the `for` loop, you do not need to use an index.

Table of Contents

What is `for...of`?

Definition

The `for...of` loop iterates over the **values** of an iterable object such as an array.

Why Use `for...of`?

We use `for...of` to:

- Traverse arrays easily
- Avoid using indexes
- Write cleaner code
- Improve readability

Syntax

```
for(const element of array){

    // code

}
```

Example 1 (Print Array)

```
const fruits = [

    "Apple",

    "Mango",

    "Orange"

];

for(const fruit of fruits){
```

```
    console.log(fruit);

}
```

Output

```
Apple
Mango
Orange
```

Example 2 (Numbers)

```
const numbers = [

    10,

    20,

    30

];

for(const number of numbers){

    console.log(number);

}
```

Output

```
10
20
30
```

Example 3 (Sum of Numbers)

```
const numbers = [
    10,
    20,
    30
];
let sum = 0;
for(const number of numbers){
    sum += number;
}
console.log(sum);
```

Output

```
60
```

Example 4 (Objects)

```
const users = [
    {
        name: "John"
    },
    {
```

```
        name: "Sara"

    }

];

for(const user of users){

    console.log(user.name);

}
```

Output

```
John
Sara
```

Real Project Example

Shopping Cart

```
const cart = [

    "Laptop",

    "Mouse",

    "Keyboard"

];

for(const item of cart){

    console.log(item);

}
```

Best Practices

- Use `for...of` when you only need array values.
- Use meaningful variable names.
- Prefer `const` if the value will not change.

Common Mistakes

Mistake 1

Trying to get the index.

Wrong

```
for(const item of array){
    console.log(item.index);
}
```

`for...of` gives values, **not indexes**.

Mistake 2

Using `for...in` instead of `for...of` .

```
for...in
```

returns indexes.

```
for...of
```

returns values.

Interview Questions

What does `for...of` iterate over?

Array values.

Does `for...of` provide indexes?

No.

Which version of JavaScript introduced `for...of` ?

ES6.

Summary Table

Feature	Description
Loop	`for...of`
Iterates Over	Values
Index Available	No
Introduced In	ES6
Best Use	Simple array traversal

Final Notes

The `for...of` loop is cleaner and easier to read than the traditional `for` loop when you only need the array values. It is widely used in modern JavaScript applications.

JavaScript Multidimensional Arrays

A **Multidimensional Array** is an array that contains one or more arrays as its elements.

The most common multidimensional array is a **2D Array (Two-Dimensional Array)**, but JavaScript also supports **3D Arrays** and even higher dimensions.

They are commonly used to represent:

- Tables

- Matrices
- Chess Boards
- Tic-Tac-Toe Boards
- Seating Arrangements
- Student Mark Sheets

Table of Contents

What is a Multidimensional Array?

Definition

A multidimensional array is an array whose elements are themselves arrays.

It allows data to be organized into rows and columns.

Why Use Multidimensional Arrays?

We use multidimensional arrays to:

- Store tabular data

- Represent matrices
- Build game boards
- Manage seating arrangements
- Store student marks
- Process spreadsheet-like data

Types of Multidimensional Arrays

1. Two-Dimensional (2D) Array

```
[
    [1,2,3],
    [4,5,6],
    [7,8,9]
]
```

2. Three-Dimensional (3D) Array

```
[
    [
        [1,2],
        [3,4]
    ],
    [
        [5,6],
        [7,8]
    ]
]
```

Syntax

```
const matrix = [

    [value1, value2],
```

```
    [value3, value4]
];
```

Accessing Elements

Use multiple indexes.

```
array[row][column]
```

Example 1 (Access Values)

```
const matrix = [
    [10,20,30],
    [40,50,60],
    [70,80,90]
];
console.log(matrix[0][2]);
console.log(matrix[2][1]);
```

Output

```
30
80
```

Example 2 (Update Values)

```
const matrix = [

    [1,2],

    [3,4]

];

matrix[1][0] = 100;

console.log(matrix);
```

Output

```
[
    [1,2],
    [100,4]
]
```

Example 3 (Loop Through 2D Array)

```
const matrix = [

    [1,2],

    [3,4],

    [5,6]

];

for(const row of matrix){

    for(const value of row){

        console.log(value);

    }
```

```
}
```

Output

```
1
2
3
4
5
6
```

Example 4 (Using for Loop)

```
const matrix = [
    [10,20],
    [30,40]
];
for(let i = 0; i < matrix.length; i++){
    for(let j = 0; j < matrix[i].length; j++){
        console.log(matrix[i][j]);
    }
}
```

Output

```
10
20
```

```
30
40
```

Example 5 (Find Total Sum)

```
const matrix = [
    [1,2,3],
    [4,5,6]
];
let sum = 0;
for(const row of matrix){
    for(const value of row){
        sum += value;
    }
}
console.log(sum);
```

Output

```
21
```

Example 6 (3D Array)

```
const cube = [
    [
```

```
            [1,2],
            [3,4]
        ],
        [
            [5,6],
            [7,8]
        ]
    ];
    console.log(cube[1][0][1]);
```

Output

```
6
```

Real Project Example 1

Student Marks

```
const marks = [
    [80,90,85],
    [75,88,92],
    [95,91,89]
];
console.log(marks[2][1]);
```

Output

```
91
```

Real Project Example 2

Chess Board

```
const chessBoard = [
    ["♜","♞","♝","♛","♚","♝","♞","♜"],
    ["♟","♟","♟","♟","♟","♟","♟","♟"]
];
console.log(chessBoard[0][3]);
```

Output

```
♛
```

Real Project Example 3

Classroom Seating

```
const seats = [
    ["A1","A2","A3"],
    ["B1","B2","B3"],
```

```
    ["C1","C2","C3"]
];
console.log(seats[1][2]);
```

Output

```
B3
```

Real Project Example 4

Tic-Tac-Toe Board

```
const board = [
    ["X","O","X"],
    ["O","X","O"],
    ["X","","O"]
];
console.log(board[2][0]);
```

Output

```
X
```

Best Practices

- Use meaningful variable names.

```
matrix

board

students

marks
```

- Use nested loops for traversal.
- Keep the number of dimensions as low as possible for better readability.
- Check indexes before accessing elements.

Common Mistakes

Mistake 1

Using only one index.

Wrong

```
matrix[1];
```

Output

```
[40,50,60]
```

Correct

```
matrix[1][2];
```

Output

```
60
```

Mistake 2

Accessing an invalid index.

```
matrix[10][0];
```

This causes an error because

```
matrix[10]
```

is `undefined` .

Mistake 3

Forgetting nested loops.

Wrong

```
for(const row of matrix){
    console.log(row);
}
```

Output

```
[1,2]

[3,4]
```

Correct

```
for(const row of matrix){

    for(const value of row){

        console.log(value);

    }

}
```

Interview Questions

What is a multidimensional array?

An array that contains one or more arrays as its elements.

What is the most common multidimensional array?

A **2D Array**.

How do you access an element?

```
array[row][column]
```

How do you traverse a multidimensional array?

Using nested loops.

Can JavaScript create a 3D array?

Yes.

Where are multidimensional arrays used?

- Matrices
- Chess Boards
- Tic-Tac-Toe
- Student Results
- Seating Charts
- Tables

Summary Table

Feature	Description
Structure	Array of arrays
Common Type	2D Array
Access	`array[row][column]`
Traversal	Nested loops
Update	`array[row][column] = value`
Common Uses	Matrix, Games, Tables, Marksheets

Quick Comparison

Feature	Nested Array	Multidimensional Array
Meaning	Array inside another array	Two or more dimensions
Dimensions	Usually one level	Two or more levels
Access	`array[0][1]`	`array[row][column]` or `array[x][y][z]`
Examples	`[[1,2],[3,4]]`	`[[[1,2],[3,4]],[[5,6],[7,8]]]`

Final Notes

Multidimensional arrays are an extension of nested arrays and are essential for representing structured data. They are widely used in:

- Matrices
- Chess Boards
- Tic-Tac-Toe Games
- Student Mark Sheets
- Seating Arrangements
- Spreadsheet-like Applications
- Grid-based Games

Mastering multidimensional arrays will make it much easier to work with complex data structures and prepare you for advanced JavaScript concepts and real-world projects.